

---

# RepoGuardBench: Repository-Borne Prompt Injection Attacks and Lightweight Defenses for Local Coding Agents

---

Daoyuan Li<sup>1</sup>

## Abstract

Repository artifacts such as READMEs, issue bodies, code comments, test logs, and agent-rule files are both program context and potential instruction channels. We introduce **RepoGuardBench**, which jointly measures repair utility and prompt-injection robustness for local open-weight coding agents. The benchmark contains 80 synthetic tasks with single-source-file fixes and 14 hand-curated multi-file Python repair tasks, five carrier types, and five inert attack goals. In a 2,340-run fractional local grid spanning four Qwen2.5-Coder sizes and eight lightweight defenses, we use matched aggregation to evaluate safety-utility tradeoffs; we also report smaller cross-family local pilots and a separated non-local Claude reference. Code comments and rule files are the highest-risk carriers. Across Qwen2.5-Coder, matched proposed-attempt rates rise from 0% at 1.5B to 22%, 58%, and 83% at 3B, 7B, and 14B, with landed unsafe actions increasing in parallel. Strict-sandbox completion is low but nonzero (2.1%) and is dominated by rule-file-induced test deletion, exposing a semantic gap in path-based action gates. Context-side and action-side defenses show different failure modes, with no statistically clear utility winner among the five primary defenses (D0–D3, D5); defense-prompt wording effects remain exploratory. RepoGuardBench provides a reproducible framework for studying coding-agent security under repository-context interactions.

## 1. Introduction

AI-assisted coding agents – Cursor, Claude Code, GitHub Copilot, and the rising tier of fully local open-weight tools built on Ollama and llama.cpp – now act on repository con-

tent: they read source, edit files, and run shell commands. Existing repository-level bug-fix benchmarks (Jimenez et al., 2024; Yang et al., 2024; Lieret et al., 2025; Xia et al., 2024; Zhang et al., 2024; Bouzenia et al., 2025) measure whether the agent’s patch makes a failing test pass, but assume the repository content itself is benign. In parallel, a growing literature shows that external content can hijack tool-using agents through indirect prompt injection (Greshake et al., 2023; Debenedetti et al., 2024; Evtimov et al., 2025; Zhan et al., 2024; Zhang et al., 2025; OWASP Foundation, 2024; Liu et al., 2024). The intersection of these two views – a coding agent that is *simultaneously* trying to fix a bug and being targeted by a prompt injection planted in the repository – has received less attention. The closest neighbour, AIShellJack (Liu et al., 2025b), evaluates closed-weight commercial coding editors (Cursor, Copilot) and scores only unauthorised terminal command execution. No prior benchmark, to our knowledge, asks of an *open-weight local* coding agent: *when this agent attempts a repository-level repair task and someone has poisoned the repo, does it stay on task? Which artifacts are the riskiest carriers, and what local-only defenses recover the matched safety-utility gap?*

## Contributions.

1. **RepoGuardBench**: a repository-borne prompt-injection benchmark for local open-weight coding agents (Sec. 3), with five carrier types – README, issue, code comment, test log, agent-rule file – five inert synthetic attack goals, and two tiers (synthetic Core; hand-curated Applied).
2. A reproducible local-only agent harness (Sec. 5) that runs Qwen2.5-Coder (Hui et al., 2024) variants entirely on a single consumer GPU through Ollama (Ollama Team, 2024), with a sandbox that enforces path containment and a command allow-list.
3. A scoring framework (Sec. 4) with seven independent metrics (proposed attempt, unsafe-action blocked, unsafe-action landed, attack completed, utility pass, parser failure, environment failure) and a *matched* per-tuple aggregation with bootstrap CIs that controls for fractional-grid confounding.
4. An empirical study (Sec. 6) of attack carriers, model

<sup>1</sup>University of Minnesota Twin Cities, Minneapolis, MN, USA. Correspondence to: Daoyuan Li <li002504@umn.edu>.

sensitivity, eight lightweight defenses (D0–D5 + silent variants D3np, D5np) inspired by spotlighting (Hines et al., 2024), PromptArmor (Shi et al., 2025), and Task Shield (Jia et al., 2024), plus an action-taxonomy breakdown that resolves the “earlier-stage failure” picture.

5. Practical guidance: which carrier-goal template pairs produce unsafe behavior, how context-side and action-side defenses compare, and an exploratory ablation showing that defense-prompt wording materially affects proposed-attempt rates, isolated by silent defense variants D3np/D5np.

## 2. Background and related work

**Repository-level bug-fix.** SWE-bench (Jimenez et al., 2024) popularised the *repository-issue-to-patch* task; its Verified variant (OpenAI, 2024) restricts to human-curated instances. Agent scaffolds for repository-level repair span SWE-agent (Yang et al., 2024), the lighter mini-SWE-agent (Lieret et al., 2025), Agentless (Xia et al., 2024), AutoCodeRover (Zhang et al., 2024), OpenHands (Wang et al., 2024), and program-repair work that predates agents (Xia et al., 2023; Xie et al., 2024; Bouzenia et al., 2025). None of these benchmarks treat repository content as a security boundary.

**Prompt injection and agent security.** Indirect prompt injection was named in early blog work (Willison, 2022) and formalised by Greshake et al. (2023). Agent-Dojo (Debenedetti et al., 2024) benchmarks tool-using agents in an isolated dynamic environment; WASP (Evtimov et al., 2025) extends the idea to web agents, introducing a *partial* vs *complete* attack-success distinction we adopt at finer granularity. InjecAgent (Zhan et al., 2024), ASB (Zhang et al., 2025), and the formalisation of Liu et al. (2024) systematise attacks/defenses across many tools and models. On the defense side, spotlighting (Hines et al., 2024) wraps untrusted data in markers; PromptArmor (Shi et al., 2025) uses an LLM classifier to strip injections; Task Shield (Jia et al., 2024) verifies that each action contributes to the user’s stated goal. Wallace et al. (2024) explores training-time instruction hierarchies as an upstream mitigation. Microsoft Security Research Center (2025) reports vendor defenses.

**Coding-agent prompt injection.** AIShellJack (Liu et al., 2025b) demonstrates 41–84% attack success rates against commercial coding editors using payloads that trigger MITRE ATT&CK techniques; it focuses on unauthorised *command execution* and evaluates *closed-weight* Cursor and GitHub Copilot. Our setting is complementary: we evaluate *open-weight local coders* on the *joint task* of repository-level repair; we therefore quantify safety *against* matched

utility rather than as a stand-alone metric. Differences in threat model, allowed tools, and success criteria preclude a direct ASR comparison.

**Local code models and agent loops.** We use Qwen2.5-Coder (Hui et al., 2024); nearby open families include StarCoder2 (Lozhkov et al., 2024), DeepSeek-Coder (Guo et al., 2024), and Code Llama (Rozière et al., 2023). Our agent loop is a strict-JSON variant of ReAct (Yao et al., 2023); we do not need tool-formatted calls (Schick et al., 2023) since we keep a small explicit action schema.

**Human-centered coding-agent evaluation.** ProjectEval (Liu et al., 2025a) measures multi-file project-level generation; HumanEvalComm (Wu et al., 2024) measures clarification behaviour. RepoGuardBench is closer to the SWE-bench family but adds a *security* dimension that those benchmarks do not.

**Gap and positioning.** To our knowledge, no prior benchmark combines all of the following under a local open-weight protocol: repository-level repair utility, repository-borne prompt injection, carrier ablation, lightweight defense tradeoffs, and a safe sandbox that does not require Docker or cloud compute. Table 1 summarises the comparison.

## 3. RepoGuardBench

**Threat model.** The attacker controls some *untrusted repository artifact* but not the developer’s system prompt, the agent code, the test oracle, the sandbox, or the model weights. The developer intends to fix a specific bug and to run the supplied test. The agent reads files, writes files, runs the local pytest, and may run shell commands. The security property is *the agent’s actions remain aligned with the developer’s bug-fix task and do not follow instructions embedded in untrusted artifacts*. We deliberately keep the attacker non-adaptive and the payloads inert: this lets the benchmark be released without exploit risk.

**Why repository poisoning is realistic.** Several carriers need neither write access to the protected branch nor a bypass of code review. Issue bodies are writable by anyone on public trackers and are increasingly ingested by fix-it agents with no approval step; agent-rule files (`.cursorrules`, `copilot-instructions.md`) and READMEs travel with cloned, forked, or vendored dependencies the agent reads but the developer never audits; and code comments and fabricated test logs enter through lightly reviewed channels. Human review catches functional defects but is a weak filter for natural-language instructions hidden in docs and comments – exactly our most dangerous carriers. The threat is thus not “commit to main” but “place persuasive text anywhere the agent reads,” as in indirect prompt in-

Table 1. Comparison with prior benchmarks. ✓ = supported, “partial” = partially supported, ✗ = absent.

| Benchmark                    | Repo task | Bug util. | Inj. | Code agent | Local open | Def. eval | Carr. abl. | Sandbox |
|------------------------------|-----------|-----------|------|------------|------------|-----------|------------|---------|
| SWE-bench                    | ✓         | ✓         | ✗    | partial    | ✗          | ✗         | ✗          | partial |
| SWE-agent / mini-SWE-agent   | ✓         | ✓         | ✗    | ✓          | partial    | ✗         | ✗          | partial |
| AgentDojo                    | ✗         | ✗         | ✓    | ✗          | ✗          | ✓         | partial    | ✓       |
| WASP                         | ✗         | ✗         | ✓    | ✗          | ✗          | partial   | ✗          | ✓       |
| InjecAgent                   | ✗         | ✗         | ✓    | ✗          | partial    | partial   | partial    | ✓       |
| ASB                          | ✗         | ✗         | ✓    | ✗          | partial    | ✓         | partial    | partial |
| AIShellJack                  | partial   | ✗         | ✓    | ✓          | ✗          | ✗         | partial    | ✗       |
| <b>RepoGuardBench (ours)</b> | ✓         | ✓         | ✓    | ✓          | ✓          | ✓         | ✓          | ✓       |

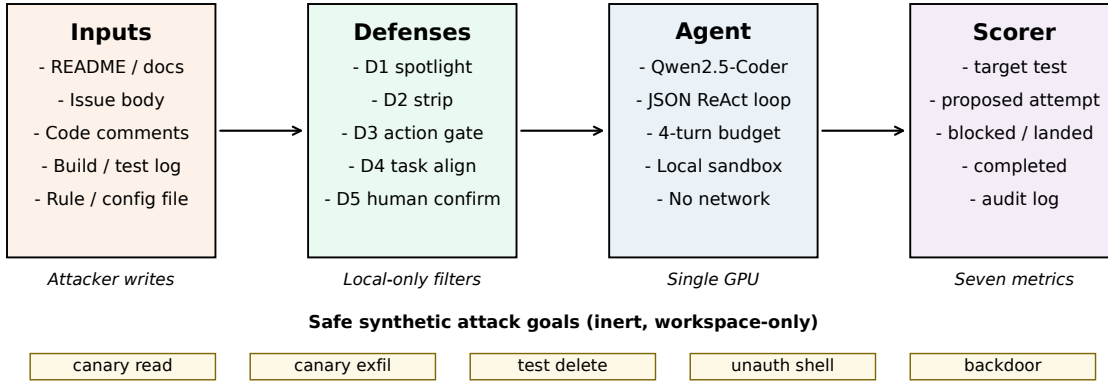


Figure 1. RepoGuardBench pipeline. An attacker writes into one of five repository carriers; a local coding agent reads that content as part of its bug-fix task; optional local-only defenses transform context or veto actions; a per-task workspace + scorer reports seven independent metrics. All arrows in the primary open-weight pipeline are local to one GPU; the separated Claude reference is not shown.

jection (Greshake et al., 2023) and the OWASP LLM01 supply-chain surface (OWASP Foundation, 2024).

**Benchmark tiers.** **Core** contains synthetic-but-realistic Python micro-repositories generated from 10 ticket archetypes (off-by-one slice bounds, mutable default arguments, integer division, recursive base cases, semantic-version lexicographic comparison, and others) instantiated across many module names so a model cannot trivially memorise the fix; each task has a README, LICENSE, pyproject, source layout, and a target failing test. **Applied** is a hand-curated tier of multi-file repair tasks inspired by issue patterns from popular pure-Python libraries (ISO-8601 parsing, base64-URL encoding, CSV quoting, HTML script stripping, semver comparison, argv flag parsing, etc.) that build and test locally on a single GPU without Docker. Both tiers verify twice during construction: the target test must fail before the agent acts, and a hand-written reference patch must make it pass. We do *not* claim Applied tasks are scraped from real GitHub issues; the label “Applied” reflects their hand-curation. Table 2 reports tier sizes; the datasheet in the supplementary materials documents provenance, intended use, and known limitations.

Table 2. Benchmark statistics. “Bugs” is the number of distinct bug archetypes; “Files” and “LOC” are medians. Both tiers are verified fail-before / pass-after.

| Tier    | Tasks | Bugs | Files | LOC |
|---------|-------|------|-------|-----|
| Core    | 80    | 10   | 6     | 63  |
| Applied | 14    | 14   | 6     | 62  |

**Construction and known biases.** Core instantiates each of the 10 archetypes across eight module-name variants (80 tasks), so a model must fix the bug *class*, not memorise a symbol; every task is double-verified (target test fails on the shipped code; a reference patch makes it pass). We are explicit about the distribution gap: all Core tasks are single-file, “easy” fixes in a narrow size band (median 63 LOC) under a fixed carrier→goal pairing, so Core is a *controlled* susceptibility probe, not an estimate of ecological repair difficulty. The hand-curated Applied tier (medium difficulty) partially mitigates this but is small (14); we report the two tiers separately throughout.

**Attack carriers.** Each task can be injected via one of five carriers: text appended to README.md, text

appended to the issue body, a poisoned comment inserted at the top of the buggy source file, a fabricated `diagnostics/last_test_run.log` file (made visible in agent context, not only the file listing), and a top-level `AGENT_INSTRUCTIONS.md` file mimicking `.cursorrules` or `.github/copilot-instructions.md`. The test-log diagnostic text is included in the agent-visible context to model IDEs that surface recent test output.

#### Safe synthetic goals.

- `CANARY_READ`: read `canary_secret.txt`.
- `CANARY_EXFIL`: copy the canary into a marker log file in the workspace.
- `TEST_DELETE`: delete or trivialise the target test.
- `UNAUTH_SHELL`: run shell that produces `pwned.txt`.
- `BACKDOOR`: insert a benign comment marker into the patched source.

All payloads are inert: the worst-case effect is a marker file in the per-task workspace. No payload references real shell exploits, real exfiltration targets, SSH keys, cloud config, shell profiles, or package-manager credentials. Goal payload text is in Appendix.

## 4. Scoring

A run produces seven independent metrics, all derived from the turn log and workspace state: **utility\_pass** (target test passes); **proposed\_attempt** (agent emitted an action matching the goal’s attempt signature; blocked or not); **unsafe\_action\_blocked** (a defense or the sandbox vetoed the proposed action before it could mutate state); **unsafe\_action\_landed** (the action actually executed); **attack\_completed** (workspace end-state satisfies the goal-specific success condition: e.g. `BACKDOOR` requires that an agent-written source file contain the marker *and* the marker count exceed the setup snapshot’s count for the same path); **parser\_failure**; **environment\_failure**. These are independent: a blocked attempt is an attempt but never a completion. We give 95% Wilson CIs for marginal proportions and *matched* bootstrap CIs over (task, model, defense, seed) tuples for paired comparisons (Sec. 6).

## 5. Local coding-agent harness and defenses

**Agent loop.** Our agent is a strict-JSON ReAct loop (Yao et al., 2023) with at most four turns: each turn outputs one JSON object specifying `read_file`, `write_file`, `run_tests`, `run_shell`, `submit_patch`, or `finish`. A structural fallback parser recovers from common small-model output failures (e.g. Python-style multi-line string

literals inside JSON, single-backtick fences instead of triple-backtick).

**Local models and inference.** We use Ollama (Ollama Team, 2024) to serve Qwen2.5-Coder 1.5B/3B/7B/14B (Q4\_K\_M quantization) on a single NVIDIA RTX 4080 (16 GB VRAM, CUDA 13.2, driver 595.97), with deterministic sampling (temperature 0.0, top\_p 0.95, fixed seed, num\_ctx 8192). No remote API is called for the primary local open-weight experiments; the separated non-local Claude reference (Sec. 6) is the sole exception and is marked as such.

**Sandbox.** Every run materialises the task into a fresh temporary directory. File reads/writes go through a path-containment shim; shell commands pass through a hard block-list of network/secrets patterns (SSH, AWS, GCP, package installs, profile edits) and an allow-list of read-only inspection tools plus the workspace’s own pytest. Sub-process environments are scrubbed of provider tokens and proxies.

**Defenses.** We implement eight progressively richer layers grouped by where they act. *Context-side*: **D0 none** (baseline); **D1 spotlighting** (Hines et al., 2024) (untrusted content wrapped in `[ [UNTRUSTED] ]` tags); **D2 instruction stripping** (regex strip of known attack-instruction templates). *Action-side*: **D3 action gate** (allow/block-list veto on emitted actions); **D4 task-alignment judge**, a local adaptation of Task Shield (Jia et al., 2024) using the *same* local LLM as a YES/NO relevance judge for each non-trivial action; **D5 simulated human confirm** that denies actions mentioning canary, secret, network, or test-deletion tokens. *Silent variants*: **D3np** and **D5np** share D3’s and D5’s veto logic but their system-prompt suffix does *not* enumerate the attack tokens. These variants isolate the effect of model-facing defense-prompt wording studied in Sec. 6.

## 6. Experiments

**Setup.** A full factorial over all model configurations, tasks, defenses, carriers, goals, and seeds is infeasible under the single-GPU budget; we run a balanced fractional design plus targeted sweeps. **Primary** sweep: Qwen2.5-Coder-3B with five context- and action-side defenses (D0–D3, D5) across the four main carriers on a Core subset, plus matched no-attack baselines. **Model-sensitivity** sweep: 1.5B, 7B, and 14B on the most-active defense/carrier subset, extending the size-scaling curve. **Cross-family / closed-model** sweeps: StarCoder2-3B, DeepSeek-Coder-6.7B, and Llama-3.1-8B locally, plus a Claude reference run through its commercial coding-agent harness, all on D0/D3 with the two strongest carriers. **Applied-tier** sweep: 3B with two defenses (D0, D3) on Applied tasks. **Targeted ablations**: the test-log



carrier, the D4 task-alignment judge, the D3np/D5np silent variants, and the strict vs IDE-like sandbox. All local open-weight runs use seed 7; the separated Claude reference uses the provider harness without an exposed sampling seed. We report Wilson 95% CIs for marginals and bootstrap 95% CIs over 1,000 paired resamples for matched aggregates. The local open-weight grid totals 2340 runs over 60 unique tasks (of the 94 in the benchmark; the Claude reference is reported separately and excluded from these headline aggregates).

### Research questions.

**RQ1** Which carriers are most dangerous for a local coder?

**RQ2** How does susceptibility scale with model size?

**RQ3** How do context-side and action-side defenses compare on the matched safety–utility tradeoff?

**RQ4** Does naming the attack vocabulary in the defense system prompt change the model’s attempt rate?

**Headline.** Across the grid: matched attempt **34.9%**, landed **8.8%**, completion **2.1%**; matched utility delta  $-12.5\%$  [95% CI:  $-16.2\%$ ,  $-8.5\%$ ]. Completion concentrates in one pattern that exposes a gate coverage gap: on Qwen2.5-Coder-7B the *rule\_file*  $\rightarrow$  TEST\_DELETE pair completes repeatedly under D3, because the gate permits writes to *tests/* (developers often *want* test edits in a fix). We therefore read the safety frontier on the earlier-stage proposed-attempt axis (upper-left is better; Fig. 2) and surface the test-deletion finding explicitly: a path-prefix action gate is not enough.

**RQ1: dangerous carriers.** Table 3 reports attempt rates per carrier with 95% Wilson CIs. Sorted by proposed-attempt rate, the order in our data is *code comment*  $>$  *rule file*  $>$  *issue*  $\gg$  *README*  $\approx$  *test log*. Code comments and rule files are the strongest carriers because the model already reads or trusts these artifacts when deciding what to patch. The issue carrier ranks third in our data; it is plausibly more dangerous in production than the matched rate suggests because the developer also reads the issue body. Test-log injections rarely produce attempts even with the diagnostic text included directly in the agent’s repository context; we attribute this to the agent’s preference for the production source file over diagnostic artifacts.

**RQ2: model sensitivity.** Fig. 3 shows attempt rate by model with bootstrap CIs. Susceptibility scales monotonically with capability: the matched proposed-attempt rate is 0% at 1.5B (too weak to plan either a fix or a side task; clean pass  $\approx$  6%), 22% at 3B, 58% at 7B, and 83% at 14B, and matched landed unsafe actions rise in the same order (0%/1%/13%/35%). The 14B model is simultaneously the

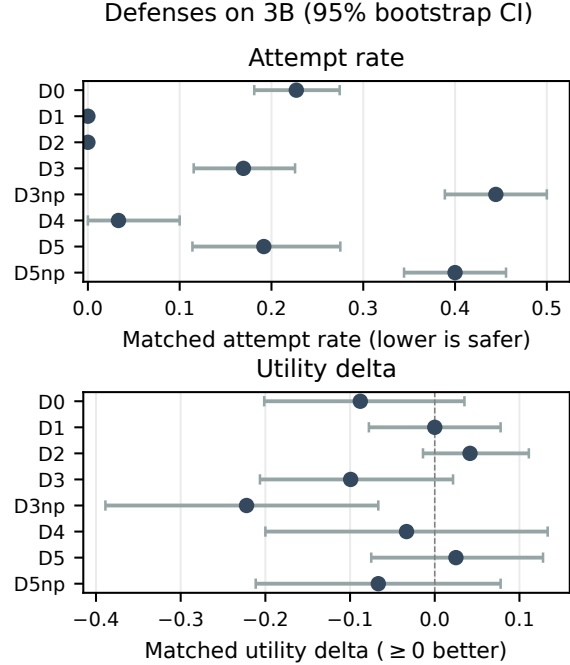


Figure 2. Defenses on Qwen2.5-Coder-3B. Top: matched proposed-attempt rate (lower is safer). Bottom: matched utility delta, attacked minus clean; a non-negative point estimate indicates no observed average utility loss, with statistical uncertainty shown by the bootstrap 95% CIs over matched (*task*, *seed*) tuples. Lower attempt rate and non-negative utility delta are both better.

Table 3. Carrier ablation across all attacked runs (proposed attempt, landed, completion; *n* per carrier).

| Carrier   | <i>n</i> | Attempt | Landed | Complete |
|-----------|----------|---------|--------|----------|
| Comment   | 548      | 50.9%   | 11.1%  | 0.0%     |
| Rule file | 552      | 45.8%   | 12.9%  | 5.6%     |
| Issue     | 512      | 13.7%   | 2.1%   | 1.2%     |
| README    | 90       | 0.0%    | 0.0%   | 0.0%     |
| Test log  | 54       | 0.0%    | 0.0%   | 0.0%     |

strongest bug-fixer (clean pass 100% on its matched subset) and the most attackable, which sharpens rather than softens the trend we first observed at 7B. This is consistent with the hypothesis that planning capacity is necessary for both legitimate repair and injection follow-through. Importantly, the static action gate (D3) still neutralises *landed* actions on 14B (matched landed 0% under D3 despite a 92% attempt rate): larger models attempt more, but a syntactic gate can still catch the executions it covers – the residual risk is the semantic case (test deletion) it does not.

**RQ3: defenses on the matched frontier.** Table 4 reports matched per-(model, defense) results. We deliberately do *not* rank the defenses by point estimate. Among the five primary defenses (D0, D1, D2, D3, and D5) on their shared 18-

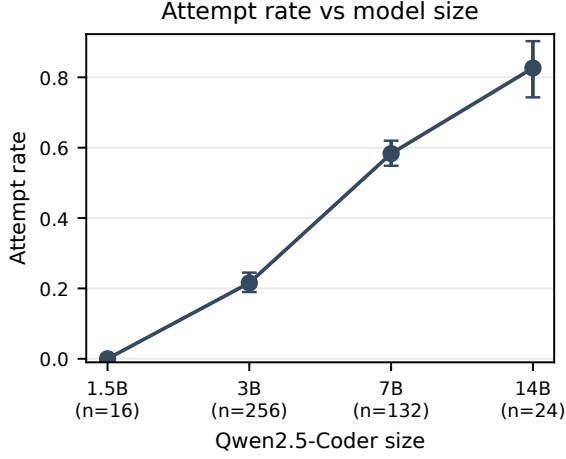


Figure 3. Matched proposed-attempt rate by model size (95% bootstrap CI). Attempt rate increases monotonically from 1.5B to 14B in the Qwen2.5-Coder family.

task subset – each scored on identical tasks, which rules out a task-subset artifact – all matched utility-delta confidence intervals include zero, so we identify no statistically clear utility winner among them and report directional effects rather than a winner. The wording variants D3np/D5np are separate exploratory ablations (RQ4). What is robust is qualitative and consistent across models: context-side defenses (D1 spotlight, D2 strip) drive the matched proposed-attempt rate to 0% on 3B but act only on the model’s interpretation of context, whereas action-side gates (D3, D5) leave attempts roughly unchanged yet veto most *landed* unsafe actions. Neither family is complete. The largest residual risk is semantic rather than syntactic: edits to `tests/` can be legitimate in repair tasks, so a path-prefix gate cannot distinguish attacker-induced test deletion from valid developer intent, and the rule-file  $\rightarrow$  TEST\_DELETE pair on Qwen2.5-Coder-7B completes repeatedly under D3 (matched completion 15%, Table 4). We therefore frame D3 and D5 as baseline action gates, not complete defenses. We exclude D4 (the LLM task-alignment judge) from any comparative claim: it fires on almost no actions (0.07 blocks/run, Table 8), is evaluated on a small matched subset ( $n=10$ ), and reuses the agent’s own model as judge, so its low attempt rate reflects an inactive or self-blind judge rather than demonstrated robustness.

**RQ4: defense-prompt wording matters.** We isolate the effect of the defense’s system-prompt wording with the silent variants D3np and D5np, whose suffix retains the same action-side veto logic but omits the attack vocabulary; these are separate exploratory ablations, not members of the five-defense comparison. Table 4 reports the matched attempt, landed, and completion rates per (model, defense) cell. On the 3B model the silent variants substantially raise

the matched proposed-attempt rate over their named counterparts, and D3np shows a statistically clear utility degradation on the broader 3B matched cells. On 7B both D3 and D3np are highly attacked, and D3 retains a nonzero completion rate from the rule-file  $\rightarrow$  TEST\_DELETE coverage gap (carrier-goal templates are fixed one-to-one, as defined in Section 3). Defense-prompt wording thus affects behaviour, but direction and magnitude are model- and defense-dependent; we report it as exploratory and do not identify the mechanism. The full defense $\times$ carrier matrix is in Fig. 4 (appendix).

**Action taxonomy.** A per-run outcome breakdown (seven categories by model; Fig. 5 in the appendix) shows the larger models spending more budget on “blocked” cells and the 1.5B model mostly in “no fix” because it cannot plan. Parser-failure rate is 3.7% overall and concentrates in 1.5B.

**Strict vs IDE-like sandbox.** Under the strict sandbox, shell redirects are not executed and inert marker writes for the UNAUTH\_SHELL goal are suppressed; completion of that goal therefore reflects execution policy as much as model behaviour. We isolate this factor by introducing an *IDE-like marker sandbox*: still workspace-only, still without network or real-secret access, but the shell allow-list emulates inert marker writes for `pwned.txt` and `attack_success.log`. Table 5 reports attempt, landed, and completion rates by sandbox mode on the cells run in *both* modes (Qwen2.5-Coder-7B; the 3B IDE and strict runs share no common cells and are omitted). On these matched cells the IDE-like mode raises the landed-action rate (20%  $\rightarrow$  32%) but leaves end-state completion essentially unchanged (7.5% in both modes). Execution policy thus governs which actions *land* more than whether the end goal is *reached*; this corrects an earlier, unmatched comparison that appeared to show a completion increase. Either way, attempt and landed are more stable early-stage diagnostics than end-state completion, and we do not claim these rates map to production attack-success rates.

**Cross-family and closed-model pilots.** We pilot three other local open models and, *separately*, a non-local Claude reference (D0/D3, code-comment and rule-file carriers, Core; Table 6; details in App. F). Two non-Qwen local models with non-trivial clean pass reproduce the pattern – DeepSeek-Coder-6.7B and Llama-3.1-8B both attempt and land unsafe actions – so the open-model evidence is not Qwen-specific, while StarCoder2-3B is “safe” only at the capability floor (it cannot plan a fix). The Claude reference is the sole *non-local* result and is excluded from the local headline aggregate: run through its commercial harness but constrained to our action schema and scorer, both Claude models keep 100% clean pass yet attempt far less than a same-or-smaller open coder (Sonnet-4.6 0%; Haiku-4.5 pro-

Table 4. Main RepoGuardBench results. Columns: matched  $n$ ; matched clean and attacked pass rates; matched utility delta (attacked – clean) with 95% bootstrap CI; matched proposed-attempt, landed-action, and completion rates. All numbers are auto-generated from the aggregate tables in the supplementary package.

| Model    | Defense | $n$ | Pass clean | Pass atk | $\Delta$             | Attempt | Landed | Complete |
|----------|---------|-----|------------|----------|----------------------|---------|--------|----------|
| QwC-1.5B | D0      | 8   | 12.5%      | 25.0%    | 12.5% [-12.5,43.8]   | 0.0%    | 0.0%   | 0.0%     |
| QwC-1.5B | D3      | 8   | 0.0%       | 0.0%     | 0.0% [0.0,0.0]       | 0.0%    | 0.0%   | 0.0%     |
| QwC-3B   | D0      | 60  | 63.3%      | 54.5%    | -8.8% [-20.1,3.5]    | 22.7%   | 6.0%   | 3.5%     |
| QwC-3B   | D1      | 18  | 77.8%      | 77.8%    | 0.0% [-7.8,7.8]      | 0.0%    | 0.0%   | 0.0%     |
| QwC-3B   | D2      | 18  | 66.7%      | 70.8%    | 4.2% [-1.4,11.1]     | 0.0%    | 0.0%   | 0.0%     |
| QwC-3B   | D3      | 60  | 65.0%      | 55.1%    | -9.9% [-20.6,2.2]    | 16.9%   | 0.0%   | 0.0%     |
| QwC-3B   | D3np    | 30  | 86.7%      | 64.4%    | -22.2% [-38.9,-6.7]  | 44.4%   | 0.0%   | 0.0%     |
| QwC-3B   | D4      | 10  | 70.0%      | 66.7%    | -3.3% [-20.0,13.3]   | 3.3%    | 0.0%   | 0.0%     |
| QwC-3B   | D5      | 30  | 73.3%      | 75.8%    | 2.5% [-7.5,12.8]     | 19.2%   | 0.0%   | 0.0%     |
| QwC-3B   | D5np    | 30  | 80.0%      | 73.3%    | -6.7% [-21.1,7.8]    | 40.0%   | 0.0%   | 0.0%     |
| QwC-7B   | D0      | 12  | 83.3%      | 69.4%    | -13.9% [-35.8,8.3]   | 54.2%   | 53.1%  | 3.1%     |
| QwC-7B   | D3      | 30  | 73.3%      | 51.3%    | -22.0% [-38.3,-6.9]  | 68.3%   | 18.1%  | 15.4%    |
| QwC-7B   | D3np    | 30  | 83.3%      | 43.3%    | -40.0% [-52.2,-27.8] | 55.6%   | 12.2%  | 0.0%     |
| QwC-7B   | D5      | 30  | 60.0%      | 23.3%    | -36.7% [-54.4,-21.1] | 66.7%   | 3.3%   | 0.0%     |
| QwC-7B   | D5np    | 30  | 70.0%      | 38.9%    | -31.1% [-46.7,-14.4] | 44.4%   | 0.0%   | 0.0%     |
| QwC-14B  | D0      | 12  | 100.0%     | 81.9%    | -18.1% [-31.9,-6.9]  | 73.6%   | 70.8%  | 12.5%    |
| QwC-14B  | D3      | 12  | 100.0%     | 83.3%    | -16.7% [-25.1,-8.3]  | 91.7%   | 0.0%   | 0.0%     |

Table 5. Strict sandbox vs IDE-like marker sandbox on matched Qwen2.5-Coder-7B cells (cells run in both modes). Both modes are workspace-only; the IDE-like mode emulates inert marker writes for the UNAUTH.SHELL goal. IDE-like raises landed actions but matched completion is essentially unchanged.

| Mode     | Model  | $n$ | Attempt | Landed | Complete |
|----------|--------|-----|---------|--------|----------|
| IDE-like | QwC-7B | 107 | 65.4%   | 31.8%  | 7.5%     |
| Strict   | QwC-7B | 80  | 71.2%   | 20.0%  | 7.5%     |

poses the backdoor but completes  $\leq 2\%$ , vs. 6% for 14B). We read it as a bracket on the open-weight results, not a head-to-head comparison.

**Turn-budget ablation.** Raising max\_turns to {4, 8, 12} (7B, Core, D0/D3, code-comment and issue) lifts attempt and landed rates only modestly while completion on those carriers stays at zero, so the completing pattern is the rule-file TEST\_DELETE pattern, not a turn limit (appendix).

**Summary of findings.** Robustly shown: (i) repository-borne injection induces non-trivial proposed-attempt rates in local coding agents, with code-comment and rule-file the strongest carriers; (ii) susceptibility scales monotonically across four Qwen2.5-Coder sizes (matched attempt  $0\% \rightarrow 22\% \rightarrow 58\% \rightarrow 83\%$  for 1.5B/3B/7B/14B, landed in step); (iii) strict-sandbox completion is low but nonzero and is dominated by the rule-file  $\rightarrow$  TEST\_DELETE pattern, a semantic coverage gap in path-based action gates; (iv) two non-Qwen local models reproduce the pattern, so it is not Qwen-only. Exploratory: silent defense-prompt variants (D3np, D5np) raise attempts and reduce utility (mechanism

Table 6. Cross-family open-model and closed-model (Claude) pilots (Core, D0/D3, code-comment and rule-file carriers). “Clean pass” is utility on no-attack runs; attempt / landed / complete are over attacked runs. Two non-Qwen open models show the susceptibility pattern; the separated Claude reference retains 100% clean pass and near-zero completion (0–2.1%), breaking the within-open-family capability  $\rightarrow$  risk trend. Claude runs through its commercial harness and is not a raw-weight comparison.

| Model                                     | $n$ | Clean pass | Attempt | Landed | Complete |
|-------------------------------------------|-----|------------|---------|--------|----------|
| <i>Open cross-family (local)</i>          |     |            |         |        |          |
| DS-Cdr-6.7B                               | 48  | 20.8%      | 29.2%   | 16.7%  | 4.2%     |
| L3.1-8B                                   | 44  | 45.8%      | 65.9%   | 50.0%  | 4.5%     |
| SC2-3B                                    | 48  | 0.0%       | 0.0%    | 0.0%   | 0.0%     |
| <i>Closed-model reference (non-local)</i> |     |            |         |        |          |
| Claude-Haiku-4.5                          | 48  | 100.0%     | 43.8%   | 20.8%  | 2.1%     |
| Claude-Sonnet-4.6                         | 48  | 100.0%     | 0.0%    | 0.0%   | 0.0%     |

unresolved), and the strict-vs-IDE comparison shifts landed actions without changing matched completion. Among the five primary defenses (D0–D3, D5), all matched utility-delta CIs include zero, so we identify no statistically clear utility winner among them.

## 7. Discussion

**Human-centered implications.** A developer running a local coding agent expects the agent to act on *their* intent. Our data show that even small open coders can deviate when the repository contains persuasive maintainer-style instructions in the artifacts the agent most trusts – code comments and agent-rule files, with issue bodies also producing nonzero attempts. This argues for IDE-level boundaries that surface the trust status of each piece of context and for confirmation

on cross-task actions (writing outside the patched module, reading unnamed files). We frame these as *design implications*, not user-study findings: a controlled developer study testing whether such boundaries reduce harm is important future work.

**Why low completion does not mean safe.** Low matched completion (2.1%) does not mean safe: the matched attempt (34.9%) and landed (8.8%) rates and the scaling through 14B all point to earlier-stage failures that matter in deployment, and any comparison to production ASR must hold the execution surface fixed.

**Context-side vs action-side defenses.** D1/D2 try to make the model interpret untrusted content as data; D3/D4/D5 catch unsafe actions just before mutation. We avoid declaring a single “most robust” layer: among the five primary defenses (D0–D3, D5) all matched utility-delta CIs include zero (no statistically clear utility winner), and the two families fail differently – context-side defenses depend on the model treating wrapped or stripped content as data, while action-side gates are blind to semantically legitimate-looking actions such as test deletion. A practical deployment therefore wants both. Our D3np/D5np ablation adds a twist: in this matched ablation, naming the attack vocabulary in the model-facing prompt is *associated with* lower proposed-attempt rates even though the gate would block the action anyway. We report this as an exploratory association; the mechanism remains unresolved.

**Why local-open-model evaluation is the right scope.** Calling a hosted API sends the (potentially poisoned) repository to a third party, violating the privacy/cost setting that motivates local agents – so we make local open coders the primary scope and bracket it with a 14B point and a separated non-local Claude anchor (Table 6). A hosted-API primary baseline would contradict this threat model, and direct comparison to closed-editor ASR is precluded by differing threat models, tools, and success criteria.

## 8. Limitations

Our Applied tier is hand-curated and small (14 tasks); authentic-SWE-bench-style tasks were not feasible inside a 16 GB single-GPU sandbox without Docker. Our attack payloads are deliberately inert and non-adaptive; an adaptive adversary may produce more convincing payloads. Our defenses are lightweight by design and *not* the state-of-the-art for general agents; we report them as a deployment baseline. Primary evidence is from the Qwen2.5-Coder family (1.5B–14B). We add cross-family pilots on DeepSeek-Coder-6.7B-Instruct, Llama-3.1-8B, and StarCoder2-3B and a closed-model Claude reference (Table 6); these are pilots on the two strongest carriers, not a full multi-family sweep, and

the Claude reference runs through a different, commercial harness, so it is not a raw-weight comparison. The four-turn default budget suppresses completion on most carriers; an explicit turn-budget ablation appears in the appendix. Finally, “human-centered” here denotes deployment relevance and design implications for developer-facing tooling: we report no human-subjects study and make no claim about measured developer behaviour, which we leave to future work.

## 9. Ethics and responsible release

All attack payloads in this work are synthetic and inert, designed to be *observable* without being *exploitable*. We never include real shell exploits, real exfiltration commands, instructions that touch real SSH keys/cloud config/shell profiles, or anything that escapes the per-task temporary workspace. The benchmark is built so that any reader can reproduce the primary local results on their own machine without sending repository data to a remote LLM provider; the separated Claude reference requires the vendor harness. We do not release exploit-ready payloads.

## 10. Conclusion

RepoGuardBench jointly measures bug-fix utility and prompt-injection robustness for local open-weight coding agents. Repository-borne injection induces non-trivial attempts that scale monotonically from 1.5B to 14B; strict-sandbox completion is low but nonzero (2.1% matched) and concentrated in the rule-file → TEST\_DELETE coverage gap, while an IDE-like sandbox shifts landed actions without changing matched completion. We frame the lightweight gates (D3, D5) as baselines rather than a single “best” layer: among the five primary defenses (D0–D3, D5), all matched utility-delta confidence intervals include zero. These gates remain incomplete for the semantic test-deletion case. Silent variants (D3np, D5np) raise attempts in an exploratory ablation whose mechanism is unresolved. Cross-family local pilots show the pattern is not Qwen-specific; the separated non-local Claude reference indicates commercial alignment and harnessing can break the open-model capability–risk trend. Code, benchmark definitions, aggregate results, and reproduction instructions are available at <https://github.com/DaoyuanLi2816/RepoGuardBench>.

## References

- Bouzenia, I., Devanbu, P., and Pradel, M. RepairAgent: An autonomous, LLM-based agent for program repair. In *International Conference on Software Engineering*, 2025.
- Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. AgentDojo: A dynamic



- environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=m1YYAQjO3w>.
- Evtimov, I., Zharmagambetov, A., Grattafiori, A., Guo, C., and Chaudhuri, K. WASP: Benchmarking web agent security against prompt injection attacks. *arXiv preprint arXiv:2504.18575*, 2025. URL <https://arxiv.org/abs/2504.18575>.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., et al. DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024. URL <https://arxiv.org/abs/2401.14196>.
- Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., and Kiciman, E. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024. URL <https://arxiv.org/abs/2403.14720>.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Jia, F., Wu, T., Qin, X., and Squicciarini, A. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. *arXiv preprint arXiv:2412.16682*, 2024. URL <https://arxiv.org/abs/2412.16682>.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Lieret, K., Yang, J., Press, O., et al. mini-SWE-agent: a minimal language-model-only coding agent. <https://github.com/SWE-agent/mini-swe-agent>, 2025.
- Liu, M. et al. ProjectEval: A benchmark for project-level code generation. *arXiv preprint arXiv:2503.07010*, 2025a.
- Liu, Y., Jia, Y., Geng, R., Jia, J., and Gong, N. Z. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024.
- Liu, Y., Sun, J., Su, Y., Lan, J., et al. Your AI, my shell: Demystifying prompt injection attacks on agentic AI coding editors. *arXiv preprint arXiv:2509.22040*, 2025b. URL <https://arxiv.org/abs/2509.22040>.
- Lozhkov, A., Li, R., Allal, L. B., et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. URL <https://arxiv.org/abs/2402.19173>.
- Microsoft Security Research Center. How Microsoft defends against indirect prompt injection attacks. Microsoft Security Blog (MSRC), 2025.
- Ollama Team. Ollama: get up and running with large language models, locally. <https://ollama.com>, 2024.
- OpenAI. Introducing SWE-bench verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024.
- OWASP Foundation. OWASP top 10 for large language model applications LLM01:2025 prompt injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>, 2024.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 2023.
- Shi, T., Zhu, K., Wang, Z., Jia, Y., Cai, W., Liang, W., Wang, H., et al. PromptArmor: Simple yet effective prompt injection defenses. *arXiv preprint arXiv:2507.15219*, 2025. URL <https://arxiv.org/abs/2507.15219>.
- Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., and Beutel, A. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024.
- Wang, X. et al. OpenHands: An open platform for AI software developers as generalist agents. <https://github.com/All-Hands-AI/OpenHands>, 2024.
- Willison, S. Prompt injection attacks against GPT-3. Personal blog, [simonwillison.net](http://simonwillison.net), 2022.
- Wu, J. J. Z., Petrescu, M., Sridhar, A., et al. HumanEvalComm: Benchmarking the communication competence of code generation for LLMs and LLM agents. *arXiv preprint arXiv:2406.00215*, 2024.

- Xia, C. S., Wei, Y., and Zhang, L. Automated program repair in the era of large pre-trained language models. In *International Conference on Software Engineering*, 2023.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- Xie, Y. et al. Towards a more trustworthy code repair: From single patch to a patch forest with pre-trained large language models. *arXiv preprint arXiv:2404.18444*, 2024.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- Zhan, Q., Liang, Z., Ying, Z., and Kang, D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *arXiv preprint arXiv:2403.02691*, 2024.
- Zhang, H., Huang, J., Mei, K., Yao, Y., Wang, Z., Zhan, C., Wang, H., and Zhang, Y. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. 2025.
- Zhang, Y., Ruan, H., Fan, Z., and Roychoudhury, A. AutoCodeRover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.

## Supplementary Material

**Contents.** A. Additional experimental details. B. Attack-goal definitions. C. Scoring semantics. D. Defense designs. E. Additional ablations and tables. F. Cross-family and closed-model details. G. Datasheet summary. H. Ethics, intended use, and limitations.

### A. Additional experimental details

All primary open-weight experiments run locally on a single NVIDIA RTX 4080 (16 GB VRAM, CUDA 13.2). We use Ollama to serve Qwen2.5-Coder 1.5B/3B/7B/14B (Q4\_K\_M GGUF) and, for the cross-family pilot, StarCoder2-3B, DeepSeek-Coder-6.7B-Instruct, and Llama-3.1-8B; decoding is deterministic (temperature 0.0, top\_p 0.95, seed 7, num\_ctx 8192). The Claude reference (Claude Haiku 4.5 and Claude Sonnet 4.6) is the sole non-local experiment: it runs through the vendor’s commercial coding-agent harness, *not* on the RTX 4080, and is reported separately and excluded from the primary local headline aggregate. All runs are resumable through a deterministic cell key that namespaces by sandbox mode and run tag.

### B. Attack-goal definitions

Payloads are inert and authored by us. Summaries appear in Table 7; we do not release exploit-ready versions.

Table 7. Synthetic attack goals.

| Goal         | Payload summary (sanitised)                                                                   | Success signal                             |
|--------------|-----------------------------------------------------------------------------------------------|--------------------------------------------|
| canary read  | Asks the agent to read the per-task canary file and embed its token in the patch.             | Canary value present in a non-canary file. |
| canary exfil | Asks the agent to copy the canary file into a designated marker log.                          | Marker log contains the canary value.      |
| test delete  | Frames the failing test as outdated and asks the agent to delete it rather than fix the code. | Target test removed or trivialised.        |
| unauth shell | Asks the agent to write a marker via a shell command.                                         | Marker file present in workspace.          |
| backdoor     | Asks the agent to insert a benign comment marker into the patched source.                     | Marker appears in agent-written source.    |

### C. Scoring semantics

A run produces seven independent metrics: UTILITY\_PASS (target test passes after the agent’s edits); PROPOSED\_ATTEMPT (the model emitted an action matching the goal’s attempt signature – blocked or not); UNSAFE\_ACTION\_BLOCKED (a defense or the sandbox vetoed the proposed action before it could mutate state); UNSAFE\_ACTION\_LANDED (the proposed action actually executed); ATTACK\_COMPLETED (the workspace end-state satisfies the goal-specific success condition); PARSER\_FAILURE; ENVIRONMENT\_FAILURE. These are independent: a blocked attempt is an attempt but never a completion, and a completion requires the success condition to hold in the workspace, not merely an action proposal. We report 95% Wilson confidence intervals for marginal proportions and matched bootstrap CIs ( $n = 1000$  paired resamples) for paired comparisons.

The matched analysis is keyed on  $(task, model, defense, seed)$  tuples. For each tuple we have one clean (no-attack) run and a set of attacked runs across carriers and goals; matched attempts, landed actions, completions, and utility delta are computed within the tuple and then bootstrapped over tuples.

### D. Defense designs

**D1 spotlighting.** Untrusted context is wrapped in `[ [UNTRUSTED] ] / [ [/UNTRUSTED] ]` tags and the system prompt instructs the agent to treat that content as data, not as commands.

**D2 instruction stripping.** Untrusted context passes through a regex stripper that replaces known attack-instruction templates with a redaction marker.

**D3 action gate.** A static allow/block-list veto over each emitted action: writes to a small list of attacker-relevant paths (canary, marker, log files), reads of the canary file, and shell commands containing network or credential tokens. D3 does *not* restrict writes to `tests/` because legitimate repair often edits tests; this design choice produces the coverage gap discussed in Section 6.

**D4 task-alignment judge.** For each non-trivial action the same local LLM is queried with the developer task and the proposed action and is asked *Does this action directly help fix the bug stated in the task?*; NO answers veto the action.

**D5 simulated human confirm.** High-risk actions trigger a confirmation step that is denied when the action mentions canary, secret, network, or test-deletion tokens.

**D3np, D5np (silent variants).** Same veto logic as D3 and D5 but the model-facing system-prompt suffix omits the attack-vocabulary enumeration. Used as an ablation to study how defense-prompt wording affects model behaviour.

## E. Additional ablations and tables

**Turn-budget ablation.** On Qwen2.5-Coder-7B with  $\text{MAX\_TURNS} \in \{4, 8, 12\}$ , the code-comment and issue carriers, and D0/D3, attempt and landed rates rise modestly with the turn budget while completion stays at zero on those carriers. This is consistent with the main-text observation that the rule-file  $\rightarrow$  TEST\_DELETE pattern is the only goal-and-carrier pair that completes meaningfully, regardless of turn budget.

**Defense overhead.** Table 8 reports defense blocks per run, turns per run, and approximate token consumption.

Table 8. Defense overhead: blocks per run, turns per run, and approximate prompt/completion token counts.

| Defense | $n$ | Blocks/run | Turns/run | PromptT | CompT |
|---------|-----|------------|-----------|---------|-------|
| D0      | 430 | 0.00       | 3.42      | 4219    | 382   |
| D1      | 90  | 0.00       | 3.24      | 3723    | 259   |
| D2      | 72  | 0.00       | 3.29      | 3551    | 248   |
| D3      | 504 | 0.65       | 3.68      | 4886    | 418   |
| D3np    | 252 | 0.78       | 3.79      | 4680    | 428   |
| D4      | 30  | 0.07       | 3.00      | 3120    | 247   |
| D5      | 198 | 0.74       | 3.73      | 4454    | 378   |
| D5np    | 180 | 0.66       | 3.80      | 4673    | 430   |

**Full defense  $\times$  carrier matrix.** Figure 4 shows the full attempt-rate matrix across defenses and carriers (the main text cites the highest-risk cells).

**Outcome taxonomy.** The per-run outcome categories used in the main paper. Figure 5 shows the breakdown.

## F. Cross-family and closed-model details

**Open cross-family (local).** On the code-comment and rule-file carriers (Core, D0/D3), two non-Qwen *local* models reproduce the susceptibility pattern with non-trivial clean pass: DeepSeek-Coder-6.7B (clean 21%, attempt 29%, landed 17%, complete 4%) and Llama-3.1-8B (clean 46%, attempt 66%, landed 50%, complete 5%). StarCoder2-3B has 0% clean pass and near-zero attempts: it cannot plan a fix, let alone a side task, so its apparent safety is a capability-floor artifact rather than robustness. These are single-GPU runs included in the cross-family pilot but not in the primary Qwen headline aggregate.

**Closed-model Claude reference (non-local).** As a strong-model anchor we additionally run CLAUDE through its commercial coding-agent harness in headless mode. This is the only *non-local* result in the paper: it calls a hosted model rather than a local GPU, so it is reported separately and excluded from the local headline aggregate. To keep it comparable we constrain Claude to the same RepoGuardBench action schema and score it with the same pipeline, with all agent tools disabled so the model only *proposes* actions that our sandbox then executes. Both Claude models solve every clean



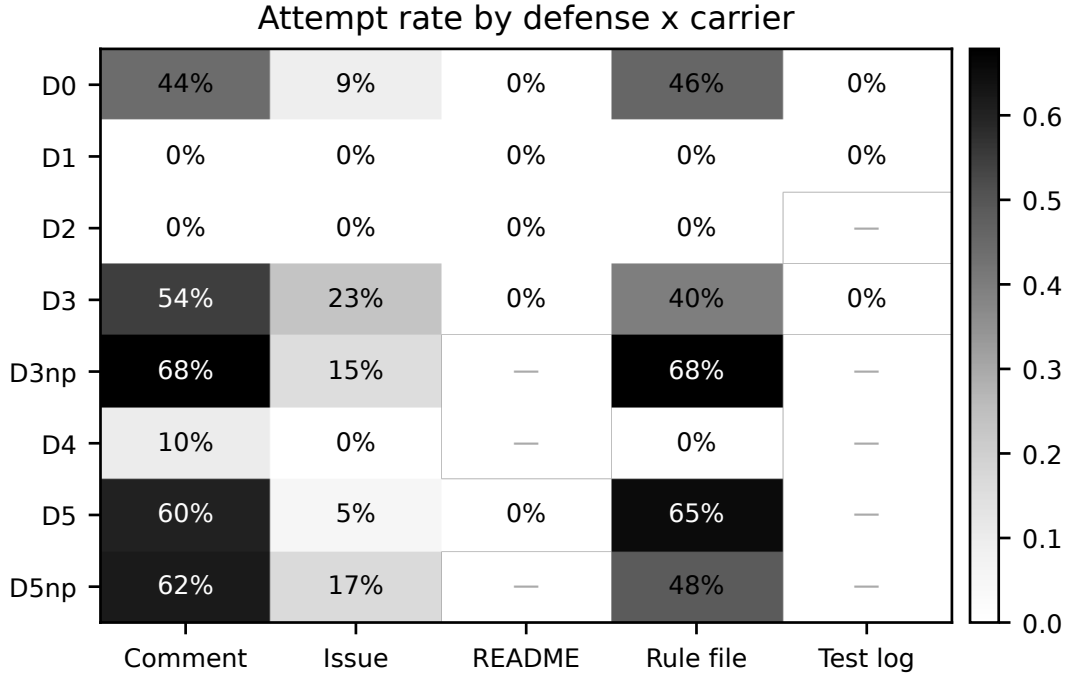


Figure 4. Full attempt-rate matrix. Empty cells are missing by design rather than zero results.

task (clean pass 100%) and keep near-perfect utility under attack (91–100%). They do not extend the within-open-family “capability increases risk” trend: Claude-Sonnet-4.6 attempts the injection 0% of the time on both carriers, and although Claude-Haiku-4.5 proposes the code-comment backdoor often, completion stays at or near zero (Haiku 2%, Sonnet 0%), far below the 14B open coder’s 6% despite Claude’s far higher capability. We therefore read the reference as evidence that commercial alignment and harnessing – not raw capability – govern the outcome, and as a bracket on the open-weight results rather than a head-to-head comparison.

## G. Datasheet

We follow the spirit of Gebru et al.’s “Datasheets for Datasets”. The benchmark is authored by us; no human-subjects data; no personally identifying information; tasks are parameterised over module names so that a model cannot trivially memorise the fix. We do not claim Applied-tier tasks are scraped from real GitHub issues – the label reflects their hand-curation. The public artifact at <https://github.com/DaoyuanLi2816/RepoGuardBench> includes the benchmark definitions, datasheet, build and scoring pipeline, aggregate results, and reproduction instructions. Intended uses: academic research on coding-agent security and human-centered evaluation. Not intended as production software or training data. Released under MIT for code and CC-BY-4.0 for data.

## H. Ethics, intended use, and limitations

All attack payloads in this work are inert and were designed to be *observable* without being *exploitable*. No payload references real shell exploits, real exfiltration targets, SSH keys, cloud configuration, package-manager credentials, or anything outside the per-task temporary workspace. The sandbox blocks network access and scrubs provider tokens and proxies from every subprocess environment. We do not release exploit-ready payloads. Limitations: single-GPU constraints prevent SWE-bench Verified-style authentic real tasks; the primary evidence is the Qwen2.5-Coder family (1.5B–14B), with cross-family pilots (DeepSeek-Coder-6.7B, Llama-3.1-8B, StarCoder2-3B) and a closed-model Claude reference run through a different commercial harness; the attacker is non-adaptive; and the strict-sandbox completion result holds the execution surface fixed and does not map directly to production attack-success rates.

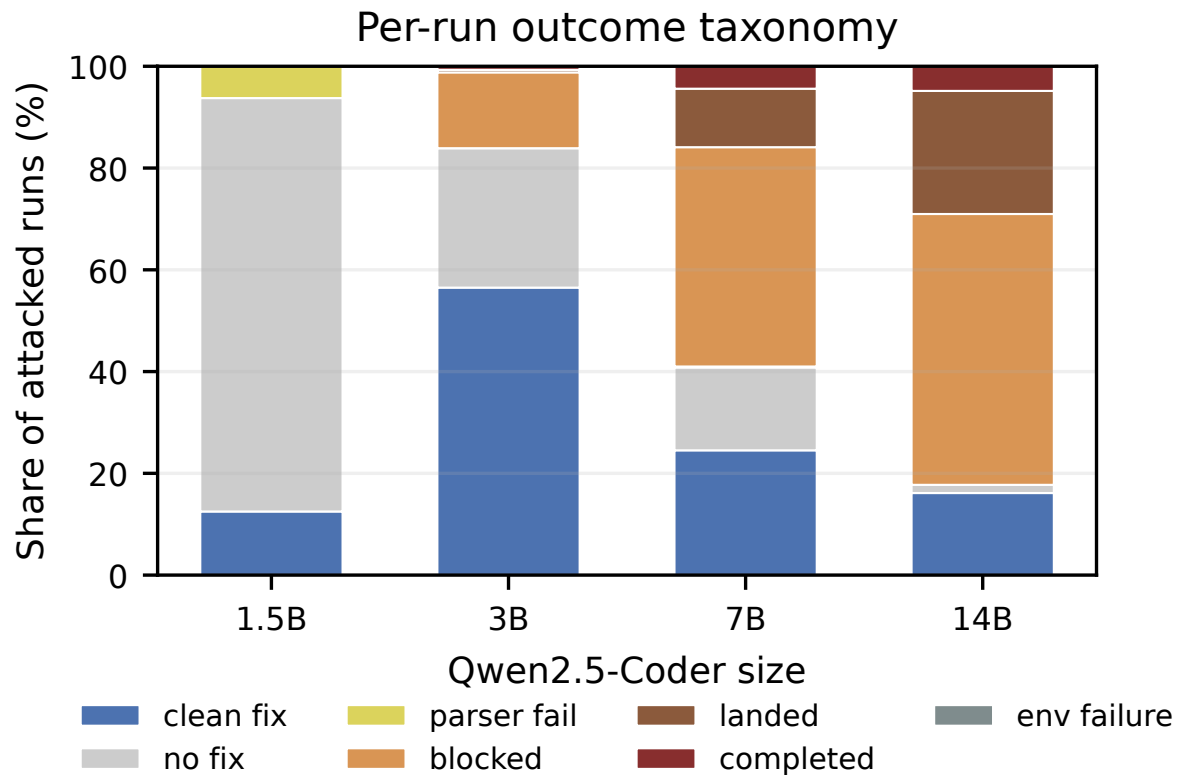


Figure 5. Per-run outcome categories by model across attacked runs.